

Vue Gadget Shop — Step 07

Informations

Notions	Composants, <code>defineProps()</code> , <code>defineEmits()</code> , <code>emit</code>
Niveau	Formatif
Travail	Individuel — sans aide IA

Objectif

Extraire l'affichage d'un gadget dans un composant `GadgetDisplay.vue` qui reçoit les données via des props et communique avec le parent via `emit`.

Résultat attendu

L'application doit fonctionner exactement comme avant — le rendu visuel ne change pas. Seule l'architecture du code change.

Vue Gadget Shop

Découvrez les derniers gadgets électroniques de haute technologie !

Mon Panier (2)

Smartphone XZ - 799€



Écouteurs Bluetooth - 199€



 Total : **998€**



Smartphone XZ

Prix : 799€

 En stock

Ajouter au panier



Laptop Pro

Prix : 1299€

 En rupture de stock

Ajouter au panier



Écouteurs Bluetooth

Prix : 199€

 En stock

Ajouter au panier

Ce qui est fourni

Structure du projet

```
/vue-gadget-shop
├── src/
│   ├── App.vue
│   ├── components/
│   │   └── GadgetDisplay.vue
├── public/
│   ├── phone.jpg
│   ├── laptop.jpg
│   └── earbuds.jpg
```

Template HTML de `GadgetDisplay.vue`

```
<div class="gadget">
  <img src="" alt="" />
  <h2>Nom</h2>
  <p>Prix</p>
  <p>Stock</p>
  <button class="add-btn">Ajouter au panier</button>
</div>
```

CSS de GadgetDisplay.vue — <style scoped>

```
.gadget {
  background: white;
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 2px 12px rgba(0,0,0,0.1);
  text-align: center;
  min-width: 220px;
}

.gadget img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  border-radius: 8px;
  display: block;
  margin-bottom: 1rem;
}

.gadget h2 {
  font-size: 1.2rem;
  margin-bottom: 0.5rem;
  color: #1a1a2e;
}

.gadget p {
  color: #666;
  font-size: 0.95rem;
}

.in-stock {
  color: #2e7d32;
  font-weight: bold;
}

.out-of-stock {
  color: #c62828;
  font-weight: bold;
}

.add-btn {
```

```
margin-top: 0.75rem;
background: #1a1a2e;
color: white;
border: none;
border-radius: 8px;
padding: 0.5rem 1rem;
font-size: 0.9rem;
cursor: pointer;
transition: background 0.2s;
}

.add-btn:hover:not(:disabled) {
  background: #2d2d5e;
}

.add-btn:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}
```

Contraintes

Dans `GadgetDisplay.vue`

- ✓ Déclarer les props avec `defineProps()` : `gadget` (Object)
- ✓ Déclarer l'événement avec `defineEmits(['add-to-cart'])`
- ✓ Créer une fonction qui émet `add-to-cart` avec le gadget en paramètre
- ✓ Lier toutes les données du template via les props
- ✓ Désactiver le bouton si le gadget est en rupture de stock

Dans `App.vue`

- ✓ Importer et utiliser `<GadgetDisplay>` dans le template
- ✓ Passer chaque gadget en prop via `v-for`
- ✓ Écouter `@add-to-cart` et appeler `addToCart`
- ✓ Supprimer le code HTML du gadget qui était directement dans `App.vue`
- ✓ Supprimer le CSS du gadget de `App.vue` — il est maintenant dans `GadgetDisplay.vue`
- ✗ Pas d'utilisation d'IA pour générer le code

💡 **Vérification** : Ouvrez Vue DevTools et vérifiez que `GadgetDisplay` apparaît bien comme composant enfant. Vérifiez que l'événement `add-to-cart` est émis lors du clic sur le bouton.